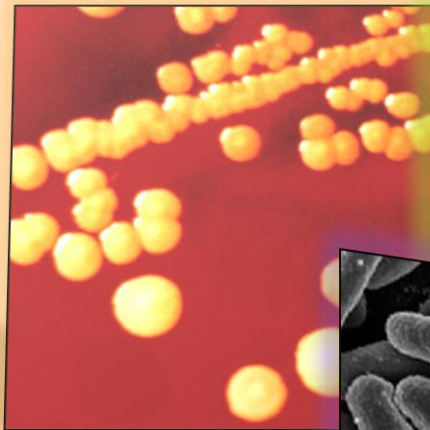


# Proyecto Final

## *Manual técnico*



Fernando Yáñez García. 318186065

Temas selectos de Ingeniería en Computación III

Proyecto Final

# Índice

|                                             |   |
|---------------------------------------------|---|
| Objetivo .....                              | 1 |
| Introducción.....                           | 1 |
| Tecnologías implementadas .....             | 1 |
| Unity .....                                 | 1 |
| Vuforia Engine.....                         | 2 |
| Visual Studio Community 2022 .....          | 2 |
| Autodesk 3ds Max .....                      | 2 |
| Github.....                                 | 2 |
| Estructura del proyecto .....               | 2 |
| Tiempo de desarrollo e implementación ..... | 4 |
| Tiempo propuesto .....                      | 5 |
| Tiempo real.....                            | 5 |
| Procedimientos .....                        | 6 |
| Configuración del proyecto .....            | 6 |
| Compilación y ejecución.....                | 6 |
| Integración de funcionalidades .....        | 6 |
| Conclusiones .....                          | 7 |

## Objetivo

Se desea la implementación de una aplicación con tecnología de Realidad Aumentada, RA, que ayude a los alumnos de educación media superior a estudiar el funcionamiento de la célula animal. Dicha aplicación estará disponible para el sistema operativo Android

## Introducción

Biología es una de las materias más interesantes entre las ciencias naturales. Elimina lo tedioso de hacer cálculos la mayoría del tiempo de la Física y contiene lo impresionante de las prácticas del laboratorio de Química (solo que menos destructivo). En temas de estudio como el ciclo de Krebs, llega a ser compleja por el sinnúmero de procesos que se llevan a cabo (y porque es un proceso más bien químico, pero se da en la célula, que tiene vida y se estudia en biología). Sin embargo, la visualización de distintas células o microorganismos en un microscopio le da ese toque interactivo de los alumnos a ella.

Es bien sabido que cada órgano de nuestro cuerpo está compuesto de células: la piel, el hígado, el corazón, etc. Podemos disponer de un pedacito de cada uno y observarlo bajo el microscopio para notar cada conjunto de células contenido en una fracción de unos milímetros cuadrados. Sin embargo, no vemos más allá de figuras deformes conectadas consecutivamente y, dentro de ellas, hay organelos (órganos) que cumplieron una función biológica. Es decir, cada célula es como un cuerpo, pero en escala muy pequeña.

La célula es la unidad estructural y funcional de todos los organismos vivos. Es como lo era el átomo, pero en biología. Todos estamos compuestos por ellas, pero no entendemos de qué se componen, cómo funcionan ni qué las hace diferentes de las vegetales, por ejemplo. A lo mejor sabremos que la mitocondria es el motor de la célula, pero hasta allí. Los alumnos de educación media superior batallan con memorizar las funciones de los organelos celulares que tienen que aprenderse antes de un examen. Para eso se desarrolla esta aplicación: una propuesta para una nueva forma de estudiar.

## Tecnologías implementadas

Para esta aplicación de RA se utilizaron herramientas de desarrollo, modelado 3D y gestión de versiones: Unity, Vuforia Engine, Visual Studio Community 2022, Autodesk 3ds Max y Github.

### UNITY

Se decidió el uso de esta herramienta por su facilidad para desarrollar aplicaciones de realidad aumentada. Cuenta con un amplio catálogo de paquetes que pueden utilizarse (como el de Vuforia o ARFoundation, ambos basados en ARCore), la facilidad para implementar animaciones en los modelos 3D importados, para manejar scripts de C# y controlar diversos comportamientos en la aplicación, etc. Además, es gratuita, por lo que se ahorra el pago de una licencia.

## VUFORIA ENGINE

Se optó por su uso dada la facilidad de implementación de detección de superficies. Se intentó la implementación con ARFoundation, pero fue más compleja y tenía muchos errores, además de ser complicado limitar la cantidad de planos detectados y la ubicación de objetos en escena. Vuforia no solamente permite la detección de superficies para ubicar los modelos 3D, sino también un recurso para poner objetos a x distancia de dicha superficie “en medio del aire”. Específicamente, se utilizó la versión 11.0.4.

## VISUAL STUDIO COMMUNITY 2022

Este IDE, además de ser compatible con Unity y permitir la actualización casi inmediata de los scripts implementados en el primero, es cómodo para utilizar y fue seleccionado dada la familiaridad con él. Aunque no se presentaron, permite la corrección de errores cuando se hace un commit en Github.

## AUTODESK 3DS MAX

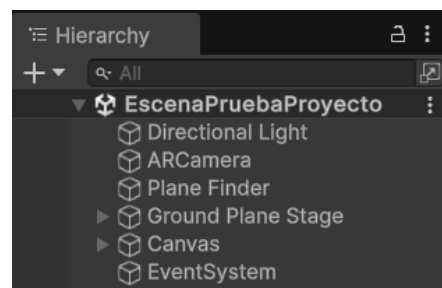
Este software de modelado en 3D es la mejor opción para el proyecto, pues un servidor está más familiarizado con él. Además, es tan dinámico que permite trabajar con modelos en formato obj, fbx y más. También, resulta más sencilla la utilización de modificadores para los vértices y caras de los modelos, el mapeo de UVs de las texturas asociadas a estos, la asignación de dichas texturas con sus correspondientes materiales y la exportación a OBJ de los modelos deseados, entre otras cosas.

## GITHUB

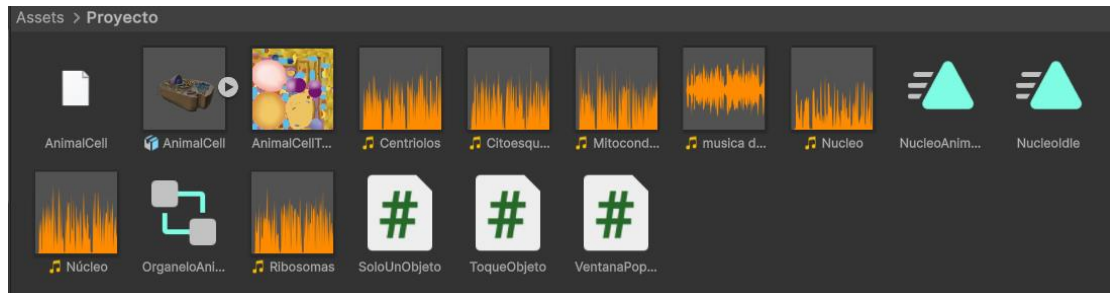
Se empleó este gestor de versiones por lo fácil que resulta usarlo y por conocerlo. Además, permite gestionar las versiones que funcionan de las que no, permitiendo hacer rollback en caso de errores tanto puntuales como catastróficos. Si bien no ocurrió alguno en el desarrollo de este proyecto, más vale prevenir

## Estructura del proyecto

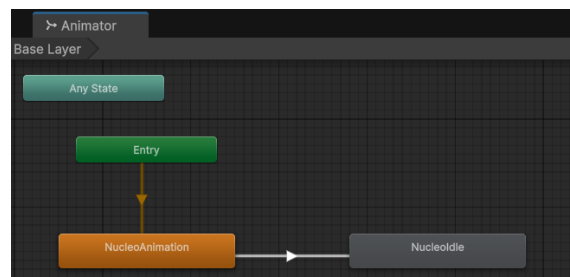
Por la parte de jerarquía, se creó una escena con base en el proyecto subido al repositorio utilizado en las prácticas de la materia. Se deja la luz direccional por defecto, se integra la ARCamera y se añaden los objetos Plane Finder, Ground Plane Stage, Canvas y EventSystem. Los dos primeros cumplen los roles de detectar la superficie donde se va a ubicar el modelo 3D determinado e instanciar dicho modelo y controlar los scripts de animación y reproducción de audios de cada organelo, respectivamente. El elemento Canvas ayuda a mostrar la información en pantalla sobre las instrucciones de ejecución de la aplicación y el audio reproducido al momento. El EventSystem se encarga de detectar las interacciones del usuario con la pantalla del dispositivo en conjunto con los scripts.



En la parte de Assets, se incluyen el modelo 3D de la célula a desplegar (separada por sus organelos), los audios de cada organelo que se puede tocar, dos audios de prueba (núcleo sin acento y música de espera), dos clips de animación para que la célula se escale o se quede quieta, un Animator Controller para determinar el comportamiento de los modelos a seleccionar y 3 scripts que controlan el despliegue de un solo objeto, el texto presente en el Canvas y el audio a reproducir tras tocar cada organelo



El funcionamiento del Animator Controller es sencillo: desde el despliegue de la célula, los organelos se encuentran en escalamiento (entre 1 y 1.1 unidades). Cuando se da un toque en la pantalla, se pasa al estado NucleoIdle, en el cual el modelo se queda estático y se puede volver a tocar, pero ya no se anima.



En el script “SoloUnObjeto” se controla la actividad del Plane Finder ya que, si se deja activado, se puede tocar cualquier punto de la pantalla para reubicar el modelo de la célula. Cuando se da el primer toque en pantalla, se desactiva el objeto PlaneFinder (planoEncontrador en la imagen) y se imprime un log en consola indicando la desactivación. Esto no se ve en tiempo de ejecución en el dispositivo, pero es efectivo

```

1  using UnityEngine;
2
3  public class DesactivarPlanoPorToque : MonoBehaviour
4  {
5      public GameObject planoEncontrador;
6
7      void Update()
8      {
9          // Detecta toque en móvil o clic en editor
10         if (Input.touchCount > 0 && Input.GetTouch(0).phase == TouchPhase.Began)
11         {
12             planoEncontrador.SetActive(false);
13             Debug.Log("PlanoEncontrador desactivado por toque.");
14         }
15         else if (Input.GetMouseButtonDown(0)) // para prueba con mouse en editor
16         {
17             planoEncontrador.SetActive(false);
18             Debug.Log("PlanoEncontrador desactivado por clic.");
19         }
20     }
21 }
22
  
```

En el script “VentanaPopUp” se controla el primer mensaje que sale tras ubicar la célula. Simplemente se le dice al usuario que seleccione el organelo que quiere estudiar.

```

1  using UnityEngine;
2  using System.Collections;
3  using UnityEngine.UI;
4  using TMPro;
5
6  // Script de Unity (1 referencia de recurso) | 0 referencias
7  public class VentanaEmergente : MonoBehaviour
8  {
9      public TMP_Text ventana; // Tu panel o ventana UI
10
11      // Mensaje de Unity | 0 referencias
12      void Update()
13      {
14          // Detecta toque en móvil o clic en editor
15          if (Input.touchCount > 0 && Input.GetTouch(0).phase == TouchPhase.Began)
16          {
17              ventana.text = "Ahora toca un organelo para estudiarlo";
18          }
19      }
20  }

```

Finalmente, en el script “ToqueObjeto” se tiene un arreglo tanto de clips de audio como de modelos 3D. En cada uno, se ubican organelo y audio en el mismo índice. Se emplean elementos tipo Ray, RaycastHit y Physics para la detección del contacto con un organelo (además de añadir un collider a los modelos). Se verifica constantemente si algún modelo del arreglo fue tocado. Si sí, se reproduce el audio y se actualiza el texto en el Canvas con el nombre del organelo cuyo audio se está reproduciendo.

```

1  using UnityEngine;
2  using TMPro; // Importa el espacio de nombres de TextMeshPro
3
4  // Script de Unity (1 referencia de recurso) | 0 referencias
5  public class SonidoVAnimacionOrganelo : MonoBehaviour
6  {
7      public GameObject[] organelos; // Array de los organelos (célula animal)
8      public AudioClip[] audiosOrganelos; // Array de AudioClips correspondientes a los organelos
9      public TMP_Text nombreOrganeloText; // Usar TMP_Text en lugar de Text (TextMeshPro)
10     private AudioSource audioSource; // Componente de AudioSource
11     private AudioClip audioActual; // El audio que se está reproduciendo actualmente
12     private Animator animador; // Componente Animator para animar los organelos
13
14     // Mensaje de Unity | 0 referencias
15     void Start()
16     {
17         audioSource = gameObject.AddComponent(); // Añadir AudioSource al GameObject que usa este script
18     }
19
20     // Mensaje de Unity | 0 referencias
21     void Update()
22     {
23         if (Input.touchCount > 0 && Input.GetTouch(0).phase == TouchPhase.Began)
24         {
25             Ray ray = Camera.main.ScreenPointToRay(Input.GetTouch(0).position);
26             RaycastHit hit;
27
28             if (Physics.Raycast(ray, out hit))
29             {
30                 GameObject objetoTocado = hit.transform.gameObject;
31
32                 // Verifica si el objeto tocado es un organelo
33                 for (int i = 0; i < organelos.Length; i++)
34                 {
35                     if (objetoTocado == organelos[i]) // Si el objeto tocado es el i-ésimo organelo
36                     {
37                         // Detener audio si ya está reproduciéndose
38                         if (audioActual != audiosOrganelos[i])
39                         {
40                             if (audioSource.isPlaying)
41                             {
42                                 audioSource.Stop();
43                             }
44                             audioSource.clip = audiosOrganelos[i];
45                             audioSource.Play();
46                             audioActual = audiosOrganelos[i]; // Actualizamos el audio actual
47                             Debug.Log("Reproduciendo audio del organelo: " + audiosOrganelos[i].name);
48                         }
49
50                         // Animar al organelo tocado (si tiene Animator)
51                         animador = objetoTocado.GetComponent<Animator>();
52                         if (animador != null)
53                         {
54                             animador.SetTrigger("Toque"); // Activa una animación al tocar el organelo
55                         }
56
57                         // Actualizar el nombre del audio en el TextMeshPro
58                         nombreOrganeloText.text = "Audio: " + audiosOrganelos[i].name;
59                     }
60                 }
61             }
62         }
63     }
64 }

```

## Tiempo de desarrollo e implementación

Para efectos de este documento, se considerarán dos cronogramas temporales: el último determinado en la propuesta de proyecto y el real, con sus respectivas fechas.

## TIEMPO PROPUESTO

| Actividad                                           | Fecha de inicio prevista | Fecha de inicio real | Fecha final prevista | Fecha final real |
|-----------------------------------------------------|--------------------------|----------------------|----------------------|------------------|
| Propuesta de proyecto                               | 7/04/2025                | -                    | 7/04/2025            | -                |
| Aprobación de proyecto                              | 7/04/2025                | -                    | 7/04/2025            | -                |
| Cambios discutidos tras aprobación                  | 7/04/2025                | 5/05/2025            | 9/04/2025            | 5/05/2025        |
| Búsqueda de modelos                                 | 11/04/2025               | 6/05/2025            | 25/04/2025           | 7/05/2025        |
| Modelado de elementos                               | 11/04/2025               | 7/05/2025            | 25/04/2025           | 8/05/2025        |
| Fusión de modelos y elementos                       | 25/04/2025               | 8/05/2025            | 30/04/2025           | 9/05/2025        |
| Diseño de UI, programación de eventos y animaciones | 30/04/2025               | 9/05/2025            | 9/05/2025            | 11/05/2025       |
| Pruebas y correcciones                              | 9/05/2025                | 11/05/2025           | 13/05/2025           | 13/05/2025       |
| Presentación del proyecto                           | 14/05/2025               | 14/05/2025           | 14/05/2025           | 14/05/2025       |

## TIEMPO REAL

| Actividad                                           | Fecha de inicio prevista | Fecha de inicio real | Fecha final prevista | Fecha final real |
|-----------------------------------------------------|--------------------------|----------------------|----------------------|------------------|
| Propuesta de proyecto                               | -                        | -                    | -                    |                  |
| Aprobación de proyecto                              | -                        | -                    | -                    |                  |
| Cambios discutidos tras aprobación                  | 5/05/2025                | 5/05/2025            | 5/05/2025            | 5/05/2025        |
| Búsqueda de modelos                                 | 6/05/2025                | 6/05/2025            | 7/05/2025            | 9/05/2025        |
| Modelado de elementos                               | 7/05/2025                | -                    | 8/05/2025            | -                |
| Fusión de modelos y elementos                       | 8/05/2025                | 12/05/2025           | 9/05/2025            | 13/05/2025       |
| Diseño de UI, programación de eventos y animaciones | 9/05/2025                | 13/05/2025           | 11/05/2025           | 15/05/2025       |
| Pruebas y correcciones                              | 11/05/2025               | 15/05/2025           | 13/05/2025           | 16/05/2025       |
| Presentación del proyecto                           | 14/05/2025               | 19/05/2025           | 14/05/2025           | 19/05/2025       |

Dados los cambios de las fechas de entrega, el cambio de tecnologías de ARFoundation a Vuforia nuevamente y haber encontrado un solo modelo 3D con todo lo de la célula, los tiempos se recortaron. Por eso mismo, se elimina el tiempo del modelado de elementos y se extiende el de búsqueda, pues se buscó un modelo accesible y atractivo.



## Procedimientos

En este proyecto se contemplan 3 procedimientos principales: la configuración del proyecto, la compilación y ejecución del proyecto y la integración de funcionalidades.

### CONFIGURACIÓN DEL PROYECTO

Para la configuración, se abre un proyecto del tipo Universal 3D en Unity con la versión de Unity 6 seleccionada. Posteriormente, se va a Edit -> Player Settings y se cambia el nombre de la compañía y del proyecto para que, al instalar el APK, la aplicación sea más fácil de encontrar. Igualmente, se configuran los parámetros de compilación por IL2CPP y contemplando la arquitectura ARM64. Finalmente, se descarga la versión 11.0.4 de Vuforia Engine SDK para la implementación de las tecnologías de RA. El proceso se detalla de mejor manera [en el siguiente video](#). Igualmente, la configuración del repositorio de Github se explica en [este video](#), aunque es sencillo: se descarga Github Desktop, se clona un repositorio, se copia el proyecto que se está trabajando en la carpeta de Github y en el Unity Hub se importa el proyecto desde el disco.

### COMPILACIÓN Y EJECUCIÓN

La compilación y ejecución del proyecto fue posible gracias a la conexión de un dispositivo móvil y sus opciones de desarrollador. Si bien se puede compilar el proyecto localmente para crear el archivo APK, Unity permite que, si hay un dispositivo conectado, el APK pueda instalarse directamente. Solo hace falta activar la depuración USB y confirmar en el dispositivo cuando se conecte y el programa, tras la compilación, carga e instala la aplicación para ejecutarla de inmediato. En casos donde se usan ImageTargets de Vuforia, la depuración se puede hacer con una webcam. Sin embargo, en el caso de la detección de suelo de este proyecto es más fácil ejecutar desde el dispositivo móvil.

### INTEGRACIÓN DE FUNCIONALIDADES

La integración de funcionalidades fue sencilla en cuanto a Canvas y recursos de texto se refiere, pues son cosas que ya se habían previsto en las prácticas. Por otra parte, la implementación de detección de suelo y animaciones fue difícil. Hay objetos 3D que incluyen sus animaciones 3D o que se les pueden adjuntar al descargarlas de internet. Sin embargo, ¿Quién ha animado una célula? Se tuvo que crear desde cero los clips de animación, la configuración de los Animator Controller y los scripts para controlar cada uno de los aspectos de detección del toque, como el trigger de la animación para que se calme cuando un organelo fue tocado, los audios con Loquendo, etc. Así mismo, se tuvo que consultar la documentación oficial de Vuforia en esta tecnología para comprender su funcionamiento y hacer una correcta implementación.



## Conclusiones

Este proyecto estuvo envuelto entre muchas actividades, tanto académicas como ajenas a la facultad. Sin embargo, se pudo concluir satisfactoriamente dado lo predeterminado en la propuesta de proyecto. Fue complicada la investigación sobre la implementación de detección de suelo, pues supuse que Vuforia no tenía recursos para ello, hasta que ARFoundation me empezó a fallar y seguí investigando. Tras ese hallazgo, el resto de la implementación prosiguió sin temas. Cuando se comenzó la implementación de animaciones, el Animator Controller causó serios dolores de cabeza, Sin embargo, tras otra ardua investigación y mucha prueba y error, se logró implementar de forma correcta.

Igualmente fue sencillo al final, pues la animación es la misma para los 5 organelos interactivos y solamente se añadieron componentes en los modelos, se relacionó dicho Animator Controller a ellos y se les asoció su audio. Este proyecto fue complicado, pero tiene detrás de sí un aprendizaje tremendo, una investigación intensa y una intención noble que lleva existiendo más de 6 años, pues la idea de esta app surgió cuando estaba en la preparatoria y mi profesora de biología sugirió la creación de una aplicación en 3D que permitiera estudiar algún tema de la materia. No me quedé con las ganas de crearla.